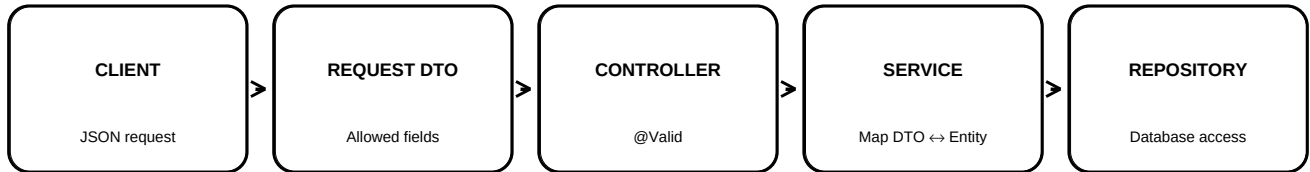


SPRING BOOT

DTOs & Validation

Lecture notes with topic-specific, hand-drawn-style diagrams



Core idea: separate the API contract from the database entity, and validate incoming data before processing it.

White paper • Black text and outlines only • Rectangular boxes • Left-to-right arrows

1. Existing Application Flow

A request commonly moves through the application layers before reaching the database. This diagram follows the left-to-right style in your reference image.



Why do we need DTOs?

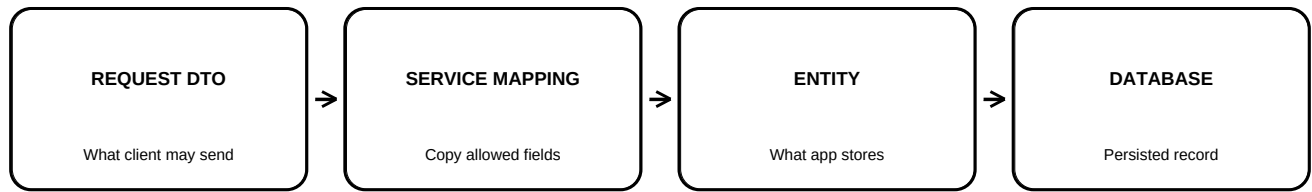
- An entity may contain fields that clients should not set or see.
- The same entity for requests and responses tightly couples the API to the database model.
- If the entity changes, the external API may change unexpectedly.

Example Student entity fields

id, name, age, email, rollNumber, subject, deleted, createdAt, updatedAt

Fields such as deleted and timestamps may be managed by the server. A client should not automatically control them just because they exist on the entity.

2. Entity vs DTO



Entity

Represents the application's persistence model. It can include client data and server-managed fields such as id, deleted, createdAt, and updatedAt.

DTO — Data Transfer Object

Carries only the fields needed for a particular API request or response. It forms a boundary between the public API and the internal entity.

Why not expose the entity directly?

- The client might try to set internal fields.
- The response might reveal fields the client does not need.
- Database changes should not automatically redefine the API contract.

Remember: DTO = API data contract. Entity = persistence model.

3. Request DTO vs Response DTO



Request DTO example

```
public class StudentCreateRequestDTO {  
    private String name;  
    private Integer age;  
    private String email;  
    private String rollNumber;  
    private String subject;  
  
    // getters and setters  
}
```

The request DTO intentionally excludes id, deleted, createdAt, and updatedAt. The server can generate or manage those values.

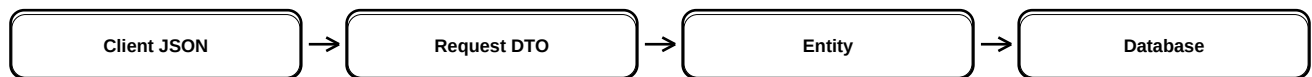
Response DTO example

```
public class StudentCreateResponseDTO {  
    private Long id;  
    private String name;  
    private Integer age;  
    private String email;  
    private String rollNumber;  
    private String subject;  
    private String message;  
    private LocalDateTime createdAt;  
    private LocalDateTime updatedAt;  
  
    // getters and setters  
}
```

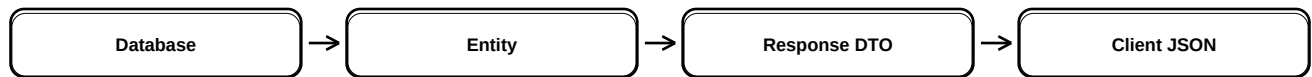
The response DTO chooses which fields are returned. It can include server-generated fields while omitting internal fields such as deleted.

4. Mapping DTO ↔ Entity

REQUEST FLOW



RESPONSE FLOW



Request DTO → Entity

The lecture initially maps fields inside the service. Only copy fields allowed by the request DTO; set server-managed values in the application.

```
private Student mapToEntity(StudentCreateRequestDTO request) {
    Student student = new Student();
    student.setName(request.getName());
    student.setAge(request.getAge());
    student.setEmail(request.getEmail());
    student.setRollNumber(request.getRollNumber());
    student.setSubject(request.getSubject());

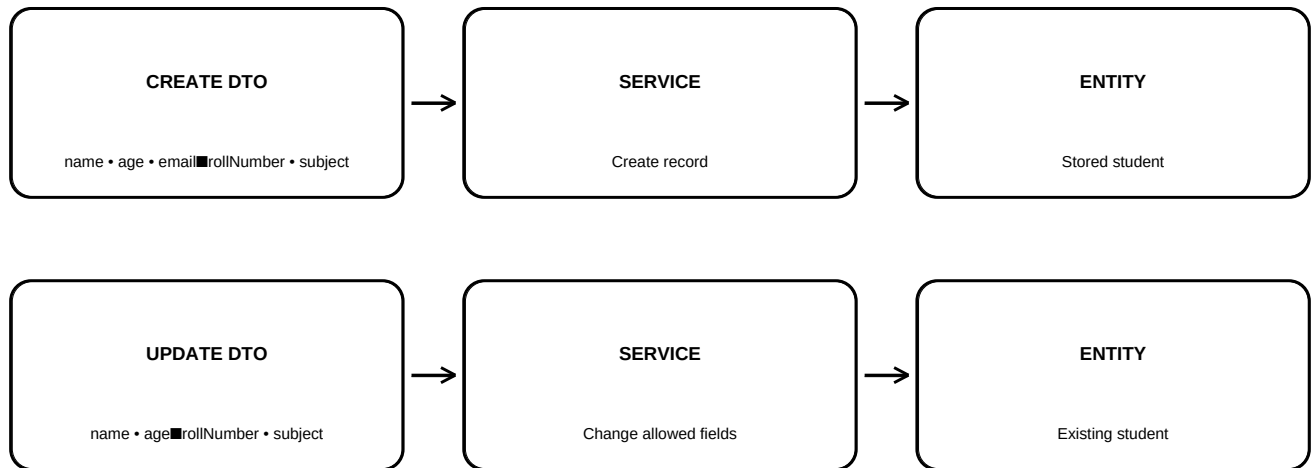
    student.setDeleted(false);
    student.setCreatedAt(LocalDateTime.now());
    student.setUpdatedAt(LocalDateTime.now());
    return student;
}
```

Entity → Response DTO

After saving or fetching the entity, copy selected values to the response DTO and return that DTO through the controller.

As the number of fields grows, mapping becomes repetitive. The lecture mentions a dedicated mapper class and possibly a builder pattern as later improvements.

5. Create DTO vs Update DTO



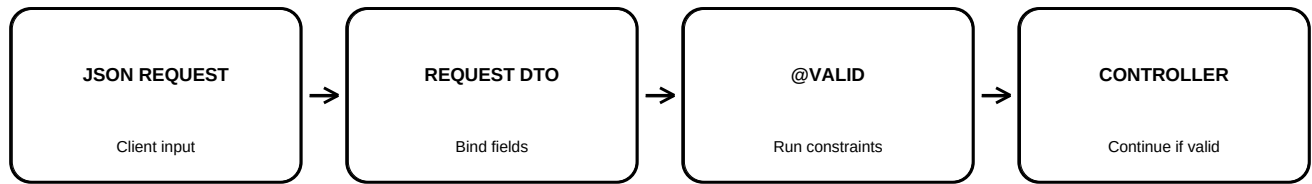
The lecture's example allows email on create but excludes it from the update DTO, so the update API does not accept email changes through that DTO.

- Different operations can have different allowed fields.
- The update service should fetch the existing entity and change only permitted fields.
- Separate response DTOs can be useful when create and update responses need different shapes.

Design rule: define a DTO around one operation's requirements rather than reusing one DTO for every operation by default.

6. What Is Validation?

Validation checks whether incoming values satisfy the rules required by the application before normal processing continues.



Rules from the lecture

- Name must be present and have an acceptable length.
- Age must be provided and be at least 18.
- Email must not be blank and should have an email-like format.
- Roll number and subject must be provided.

Put validation annotations on DTO fields. Use @Valid on the controller request parameter to trigger validation.

7. @NotNull vs @NotEmpty vs @NotBlank

For text fields, these annotations treat null, empty strings, and whitespace differently.

Value	@NotNull	@NotEmpty	@NotBlank
null	Rejects	Rejects	Rejects
"" (empty)	Allows	Rejects	Rejects
" " (spaces)	Allows	May allow	Rejects
"Kunal"	Allows	Allows	Allows

@NotNull rejects null. @NotEmpty rejects null and empty. @NotBlank also rejects whitespace-only text.

For required text like name or subject, @NotBlank is often suitable. For Integer age, @NotNull can ensure the value was provided.

8. Other Validation Annotations

Annotation	Meaning	Example
@Email	Checks email-like format	user@example.com
@Min(18)	Number must be at least 18	age = 18 passes
@Size(min=2, max=50)	Length/size in range	name length 2–50
@Positive	Value must be positive	count > 0
@Past	Date must be in the past	past date
@Future	Date must be in the future	future date

Example DTO constraints

```
@NotBlank(message = "Name is required")
@Size(min = 2, max = 50)
private String name;

@NotNull(message = "Age is required")
@Min(value = 18, message = "Age must be at least 18")
private Integer age;

@NotBlank(message = "Email is required")
@email(message = "Enter a valid email")
private String email;
```

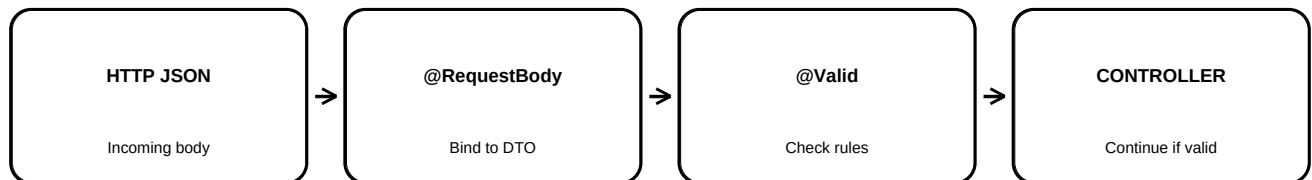
@Email checks the format, not whether the mailbox exists. Constraint messages can later be used in a custom validation-error response.

9. Using @Valid in the Controller

```
@PostMapping("/students")
public ResponseEntity<StudentCreateResponseDTO> createStudent(
    @Valid @RequestBody StudentCreateRequestDTO request) {

    StudentCreateResponseDTO response =
        studentService.createStudent(request);

    return ResponseEntity.ok(response);
}
```

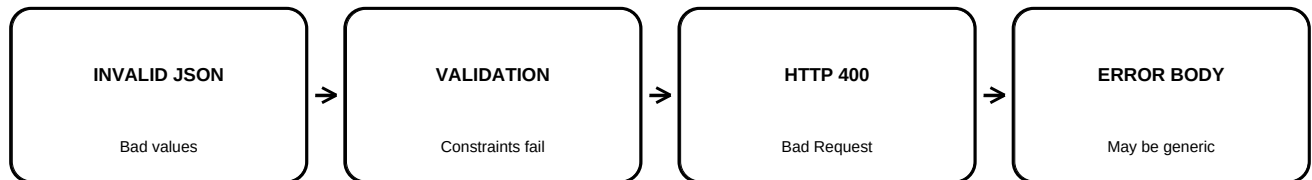


If one or more constraints fail, the request is rejected instead of continuing as a valid request. The lecture describes HTTP 400 Bad Request as the common result.

10. Invalid Data and HTTP 400

Invalid request example

```
{  
  "name": " ",  
  "age": 16,  
  "email": "not-an-email",  
  "rollNumber": "",  
  "subject": " "  
}
```

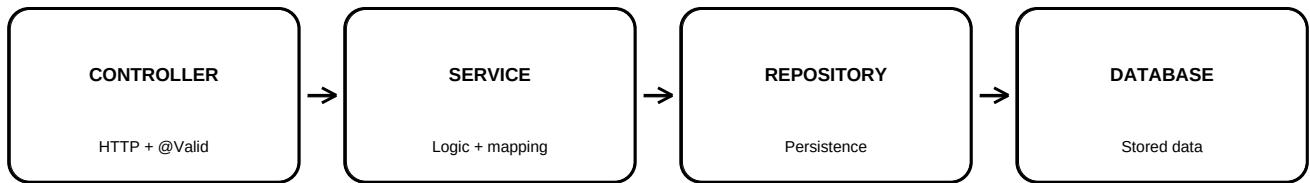


Why may the response be generic?

Spring Boot's default error response may not provide a consistent, field-specific list of validation messages. Custom exception handling is needed for a tailored response.

The transcript defers the implementation of custom exception handling to the next lecture, so this note explains the purpose without adding an unshown implementation.

11. Layer Responsibilities



- Controller: binds request JSON to a DTO, triggers validation, and calls the service.
- Service: performs business logic and initially maps DTOs to entities and entities to response DTOs.
- Repository: saves and retrieves entities.
- Database: stores the persisted records.

12. Interview Revision

What is a DTO? Data Transfer Object — a simple object for moving only the required data.

Why separate DTO and Entity? To avoid coupling the API contract to the persistence model and to control accepted/returned fields.

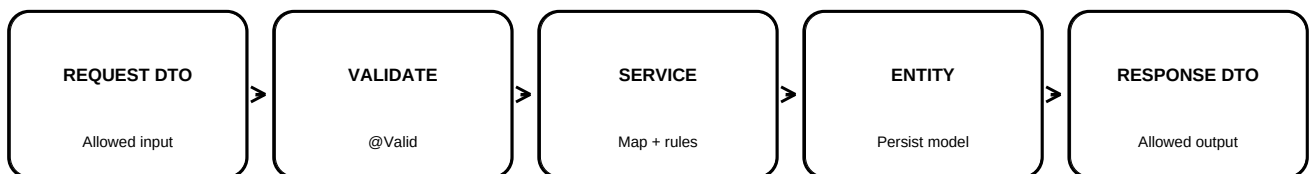
Can create and update use different DTOs? Yes, because the permitted fields can differ.

What does @Valid do? Triggers validation of the bound request DTO.

@NotNull vs @NotEmpty vs @NotBlank? Null only; null/empty; null/empty/whitespace-only for text.

Why custom exception handling? To return a consistent response with field-specific validation messages.

Final mental model



Request DTO → validate → map to entity → repository saves → map saved entity to response DTO → send JSON to client.

Prepared from your supplied lecture transcript. Examples are simplified for revision.